

Assignment Bonus

Summer Mix

AUTHOR

Operating Systems Teaching and Research Unit

PUBLISHED

14.07.2024

Hivemind Assignment

In this task, you will synchronize multiple threads to work together and calculate a result. The architecture is as follows: one main thread starts 'n' worker threads. The workers must collaborate, with each computing a value, and then report the results back to the main thread.

Code Structure

Global Structure

The global structure, `global_info`, contains information accessible to all threads, synchronization primitives, and a `main_result` structure for direct communication with the main thread.

Main Result Structure

The shared structure, `main_result`, is used for communication among all threads and must be protected by a synchronization mechanism. Each worker writes to this structure, and the main thread copies the structure and signals this action with a semaphore.

Thread Information Structure

Each thread has its own `thread_info` structure containing:

- An ID
- A secret value
- A list of “buddies” (a subset of workers)
- A `chunk` structure representing the part of the file the worker computes

Initialization Function

```
// Initialize extra info
// `glinfo` already contains n_threads.
// `path` is a path to the file to-be-processed by the workers
void init_stud_data(struct global_info* gl, char const* path) {
    // Use this function to initialize any data you added to global_info yourself.
    // Do not call it, it will be called by the main.
}
```

The function `init_stud_data` will be called a single time before the workers are started, by the main thread, to initialize the fields you have added to the structure `global_info`.

💡 Tip

More detailed information for these structures is available in the source files.

Task Overview

```
void* thread_start(void* arg) {  
    struct thread_info* tinfo = (struct thread_info*)arg;  
    // Put your code here  
    // ...  
  
    // This doesn't need to return anything  
    return NULL;  
}
```

Implement the `thread_start` function with the following specifications:

Worker Computation

Each worker computes a hash of a file chunk. You **must** use `mmap` syscall to map the file to memory. The recommended way to proceed is to call `mmap` once in the `init` function.

Each worker works on its own chunk. A chunk is divided into groups of 8 bytes, and each of these 8-byte groups is XORed together. If a chunk is not divisible by 8, handle the remaining bytes by considering them as a new chunk. Place the remaining bytes in the high-order bits and fill the low-order bits with zeros.

⚠ Warning

Converting a byte array to a `uint64_t` value is not trivial and depends on the endianness of one's system. Be careful while dealing with the subgroup of 8 bytes and ensure the byte order is correct (the same order as in the file).

Implement the `process_chunk` function with the above information.

```
uint64_t process_chunk(void const* base_ptr, struct chunk const* chk) {  
    return 0;  
}
```

Communication Between Workers

In addition to working on its chunk, each worker has a list of buddies with whom they must communicate. The final computed value of each worker is the XOR of its own chunk result, its buddies' results, and its local secret.

Synchronize with your buddies to wait until their computed values are available.

Communication with the Main Thread

A single shared structure is used for communication between workers and the main thread. This structure is shared by everyone. To facilitate communication:

- The main thread will wait on the semaphore `main_signal_result_avail` until a result becomes available.
- Once a result is available, the main thread copies the result to a local structure.
- The main thread acknowledges reading the result by posting to the semaphore `main_signal_result_read`.

Workers must synchronize with the main thread to ensure proper communication of their final computed values.

POSIX Signal Handling

Your implementation should asynchronously dump data when it receives a `SIGUSR1` signal. Write a signal handler so that every thread dumps its secret to the file described by the macro `SIGUSR1_DUMP_FILENAME`. Your implementation should create the file only if the signal is received. Otherwise, ensure the file does not exist.

All threads are writing their secret on each line of the same file. The order doesn't matter.

Note

To help you with signal handlers in C, check the documentation of `signal(2)` or `sigaction(2)`.

Complete the setup of the handler in `void setup_signal_handler()`.

Summary

A key challenge in this assignment is managing an undetermined number of threads. Some tests will use a random number of threads, so your implementation must prevent deadlocks and ensure all structures are correctly initialized. You must handle the synchronization between threads as well as the communication and synchronization between the workers and the main thread.

You can add as many fields to the global structure as needed to help with synchronization, interprocess communication, etc.

Worker Routine

The worker's routine involves these steps:

1. Each worker waits for results from a subset of other workers.
2. Each worker computes a value.
3. Each worker sends its computed value back to the main process.
4. Each worker waits for the main process to read the sent value.

 Tip

Additional tips:

- Use semaphores or mutexes to manage synchronization between threads.
- Ensure that all threads properly initialize and clean up any resources they use.
- Make sure that your implementation can work with various numbers of threads.